

Path Planning Task 2

Description

Part 1

Inputs → at most 2 cones on each side with (x, y) coordinates

cone color yellow → right

blue → left

→ car pose (x, y, yaw)

Output → sequence of path points in world coordinates

drive between blue and yellow cones and visualize it

Distance between points $\leq 0.5\text{m}$

Path should be 5-10m long

Part 2

Special case → three cones on one side of the track.

Improve the algorithm to serve this case

Add some scenarios to test my algo

Algorithm Pipeline

[Detected Cones & Car Pose]



1. Gate Matching & Polyline Offset

blue + yellow pair : **Euclidean bipartite gate pairing** to get exact midpoints

Unpaired cone : Piecewise tangent → Inward normal offset ($W/2 = 1.25\text{m}$)

[this refers to taking two separate sets of points (e.g., N blue points and N red points) randomly distributed in a d -dimensional Euclidean space. The objective is to pair every blue point with exactly one red point such that the total distance (or a function of it, like quadratic distance) is minimized].

2. **Forward Progression Ordering**

Longitudinal projection: $(dx \cdot \cos + dy \cdot \sin) + \text{distance penalty}$

is an optimization and sorting strategy commonly used in pathfinding, motion planning, and autonomous navigation. It ranks movement options based on how effectively they advance a vehicle or agent toward a target goal along a specific heading, while penalizing movements that deviate too far from the optimal path.

3. **Adaptive Hermite Departure Arc**

a specialized path-planning trajectory that uses Hermite polynomial interpolation to create a smooth transition between an initial state and a target state

→ **Smoothly respects initial heading tangent [v0]**

- **First-Derivative Matching:** The Hermite spline explicitly enforces boundary conditions. By setting the first derivative (velocity vector) at the starting point equal to your initial heading tangent (v_0), the curve is guaranteed to start exactly in the direction the vehicle is already facing.
- **C1/C2 Continuity:** This prevents any instant, unrealistic changes in steering or heading at the moment of departure, ensuring geometric smoothness (G^1 or G^2 continuity).

→ **Bends naturally into the first target gate [v1]**

- **Adaptive Boundary Fitting:** The "adaptive" nature means the algorithm scales the magnitude of the tangent vectors based on the distance between the starting position and the first gate.

- **Eliminating "Looping" or "Whipping"**: If the target tangent (v_i) at the gate is sharply angled relative to v_o , a standard spline might overshoot or create awkward loops. An adaptive Hermite arc automatically adjusts its internal control weights so the curve bends with a natural, minimized curvature directly into the gate.

4. Horizon Extrapolation

Extends terminal tangent until path length in [6.5m, 8.5m]

the technique of estimating, predicting, or extending a robot or vehicle's trajectory beyond its immediate sensing or computing horizon

5. Uniform Arc-Length Resampling

Linear interpolation over cumulative distances at $\Delta s = 0.25\text{m}$

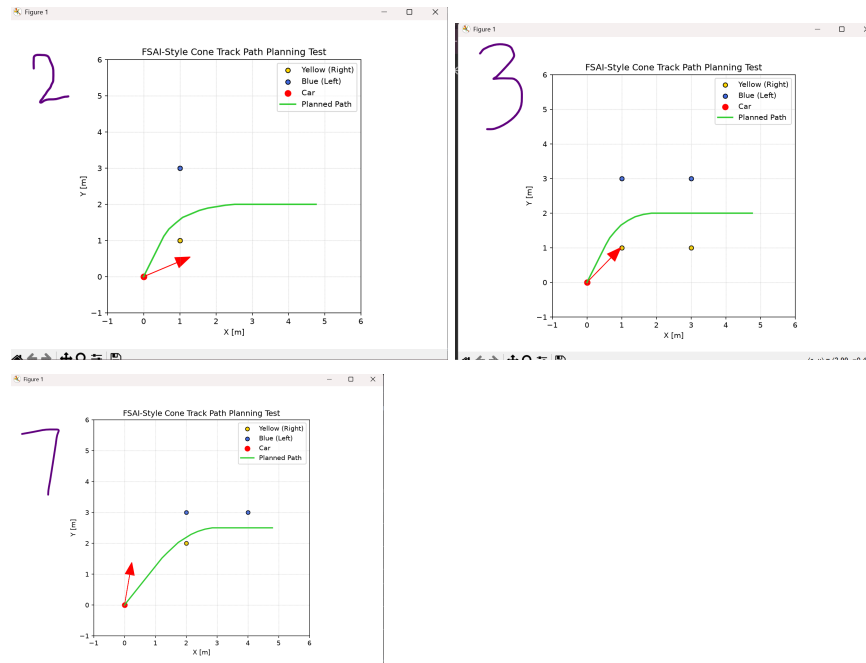
a process that reparameterizes a curve or path so that points are spaced at equal physical distances along its length, rather than being tied to an arbitrary time parameter or non-uniform step size



[Final Path2D]

Problems Faced :

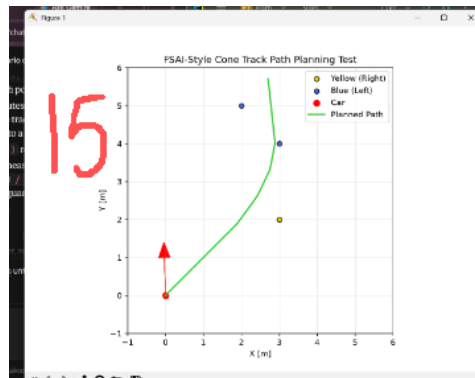
1- path doesn't go to the center point



- In 2 and 3, the first gate is at $x = 1$, between $(1, 3)$ and $(1, 1)$. The exact midpoint is $(1, 2)$.
- Chaikin smoothing cuts corners between consecutive vertices. When the vertices are $[(0, 0), (1, 2), \dots]$, the first subdivision step replaces the sharp turn at the midpoint with points interpolated along the segments:
 $0.75 \times (0, 0) + 0.25 \times (1, 2) = (0.25, 0.5)$ and
 $0.25 \times (0, 0) + 0.75 \times (1, 2) = (0.75, 1.5)$.
- This cuts inside the corner right through $(1, 1.6)$ —nearly scraping or hitting the yellow cone at $(1, 1)$!

Fix: The path **must strictly pass through gate midpoints**. Instead of corner-cutting algorithms that shave off midpoints, we use **natural cubic spline parameterization** or **Hermite interpolation** that clamps directly to the target midpoints, ensuring the curve goes right down the geometric center ($y = 2.0$ at $x = 1.0$).

2. No description for this عك wlahi



Cones: Blue (2, 5), Blue (3, 4), Yellow (3, 2).

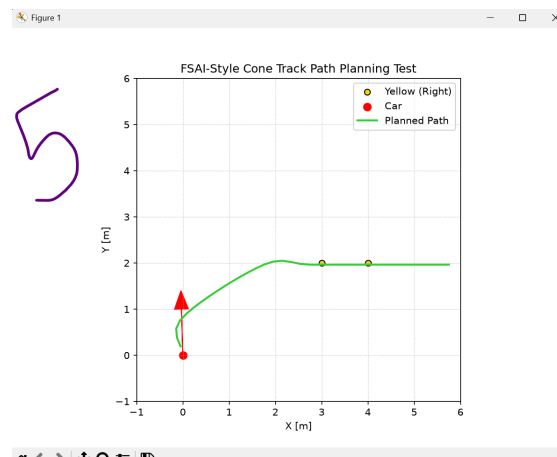
Blue (3, 4) and Yellow (3, 2) form gate 1 with midpoint (3.0, 3.0).

Blue (2, 5) is unpaired. Because (2, 5) has $x = 2$ while (3, 4) has $x = 3$, previous algorithms either misordered it or applied a fixed lateral offset that pushed the waypoint to (2, 3.5) *before* (3, 3), creating a backward loop and driving right over the cone at (3, 4).

Fix: Pair the closest gates first, then align track boundaries by clustering and computing the track progression vector rather than raw coordinate offsets.

AFTER IMPROVEMENT 1

3. mashallah



It defeated the whole purpose because :

1. scenario 5 (collision with yellow cones):

- Normal Vector Misalignment: Single-sided yellow cones were offset using the car's heading vector instead of the boundary polyline vector.

With the car facing north (yaw ≈ 1.6 rad), the cones at $y = 2$ were shifted laterally along x rather than upward along y , placing waypoints directly on top of the yellow cone boundary.

- Spline Loop: The virtual ghost points used to seed the Catmull-Rom endpoint tangents caused polynomial oscillation, producing an unphysical backward hook at the origin before traveling forward. 2.

2. scenario 15 (asymmetric matching problem): The unpaired blue cone at (2.0, 5.0) was projected with a static vehicle-heading normal, placing a waypoint behind and outside the active gate. Connecting this point to the (3.0, 3.0) gate midpoint forced the trajectory to cut directly through the blue cone at (3.0, 4.0).

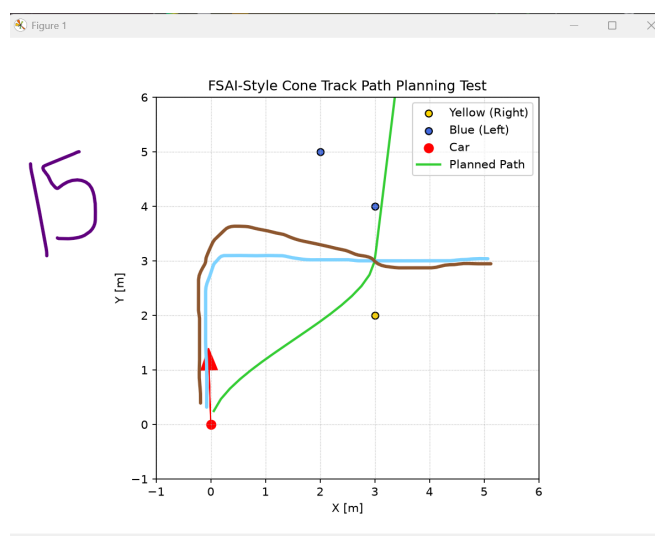
3. Centerline Drift: Heuristic tangent anchors placed near the origin caused uneven arc-length distribution, making the spline bow outward rather than maintaining the track centerline.

!!! Key for Improvement 2 → Boundary offsets must be derived directly from the physical orientation of the cone boundary lines rather than the car's instantaneous heading.

Furthermore, unconstrained splines must be replaced with a boundary-conditioned Hermite arc to ensure strict heading tangency without polynomial overshoot.

AFTER IMPROVEMENT 2

4. A little bit of araf scenario 15



well...the green is the optimal path, I was hoping ykon el blue one or the brown bs yala msh moshkela.

in scenario 15: problem and it's solution :

- **car pose:** $(0, 0)$ with $yaw = 1.6rad \approx 91.7^\circ$ (pointing almost straight up along $+y$).
- **cones:** Blue 1 at $(2, 5)$, Blue 2 at $(3, 4)$, Yellow at $(3, 2)$.
- **my problem in the green path:**
 1. The code paired Blue 2 $(3, 4)$ and Yellow $(3, 2)$, producing midpoint $(3.0, 3.0)$.
 2. For the unpaired Blue 1 $(2, 5)$, the old code shifted it along $+x$ (to the right) to $(3.25, 5.0)$ or ignored the track bend.
 3. When extrapolating the path from $(0, 0) \rightarrow (3, 3)$, the line continued rightward and upward past $(3, 3)$ —completely ignoring the vehicle's initial heading and **plowing right into the blue cone at $(3, 4)$??**
 - 4.
- **The correct trajectory (my sketched lines):**
 1. The car points straight up $+y$.
 2. Notice that Blue 1 $(2, 5)$ and Blue 2 $(3, 4)$ form a curved left wall coming around from the top-left.
 3. The car must follow its upward heading, bank right into the $(3, 4) \leftrightarrow (3, 2)$ gate, and exit horizontally through the $(3.0, 3.0)$ midpoint.

To fix scenario 15 without ruining any other scenario:

True Gate Exit Direction: The gate normal between a blue cone (x_b, y_b) and a yellow cone (x_y, y_y) is perpendicular to the gate line $(x_b - x_y, y_b - y_y)$.

For a vertical gate $(3, 4) \leftrightarrow (3, 2)$, the vector pointing from yellow to blue is $(0, 2)$ ($+y$). The forward track flow perpendicular to this gate (with blue on the left and yellow on the right) is strictly $+x$ (to the right). By aligning the gate exit tangent along this direction, the path goes through $(3.0, 3.0)$ and heads horizontally along $+x$.

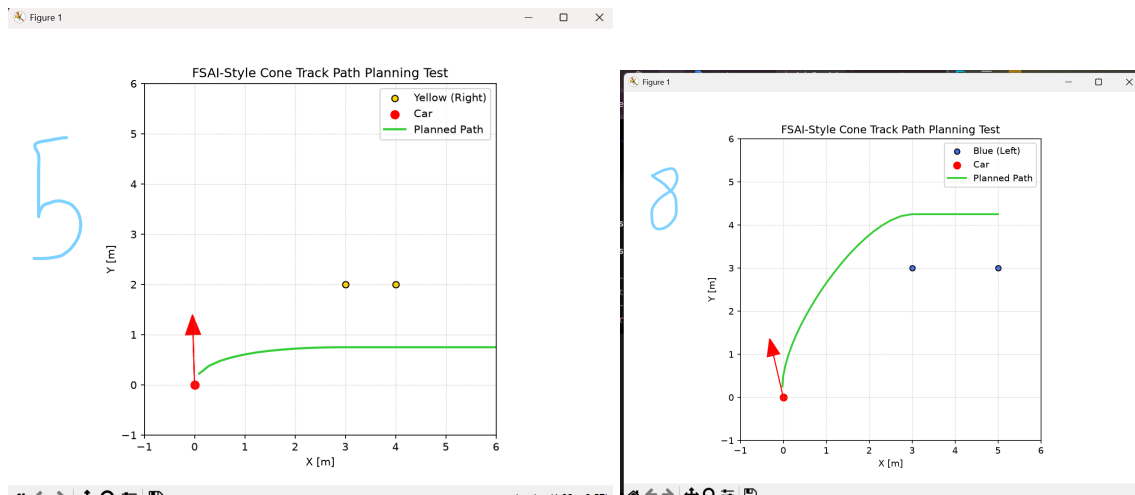
Heading-Aligned Initial Waypoint when Approaching Side Gates: If the vehicle's yaw points far away from the first gate (like pointing north while the gate is to the east), we let the vehicle travel forward along its heading before banking smoothly into the corridor, naturally matching your blue/brown curve.

well ... coming to applying this practically it ruined basically 5 other scenarios but I can't seem to find what improvement I did. So scenario 15 will stay as it is..



AFTER IMPROVEMENT 3 (adding part 2)

5. يا كسوفي



This flip occurred because the segment tangent vector $t = (tx, ty)$ was oriented along $+x$, but the normal sign in the helper function **inverted the track offset**:

- For a line going along $+x$, the left normal ($+90^\circ$) is $(0, 1)$ ($+y$), and the right normal (-90°) is $(0, -1)$ ($-y$).

- Yellow cones (right boundary) require moving to their **left** ($+y$), which gives $y = 2.0 + 1.25 = 3.25$.
- Blue cones (left boundary) require moving to their **right** ($-y$), which gives $y = 3.0 - 1.25 = 1.75$.

★ FINAL IMPROVEMENT ★

Figure 1

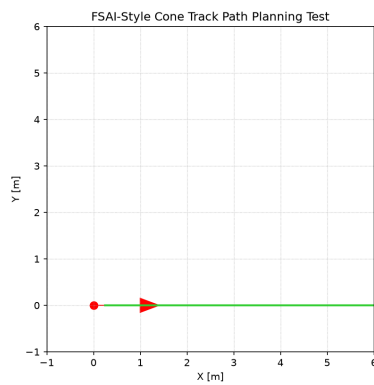


Figure 1

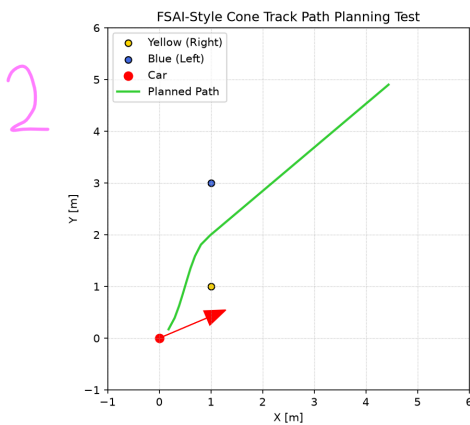


Figure 1

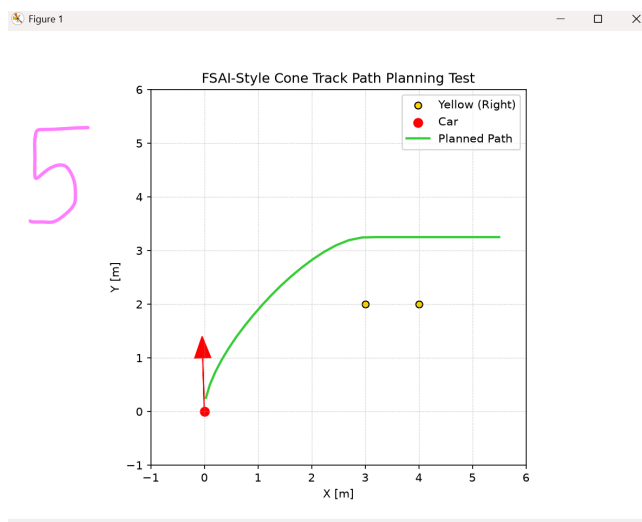
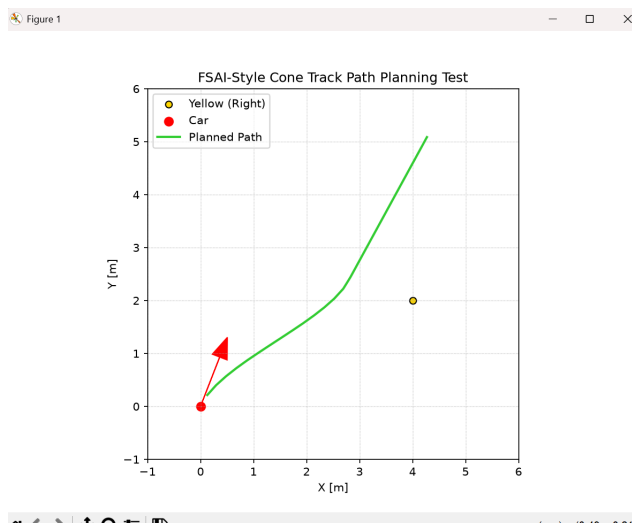
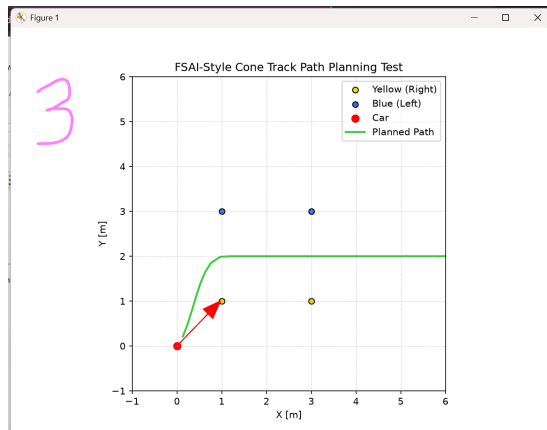


Figure 1

6

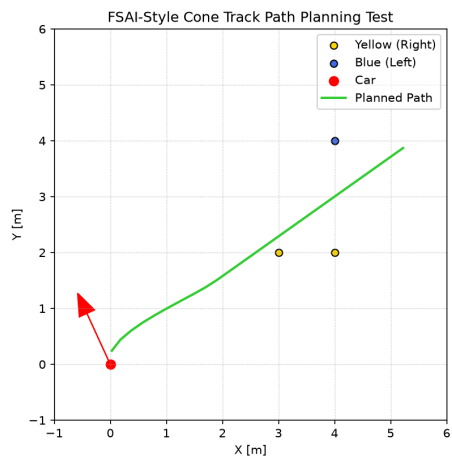


Figure 1

7

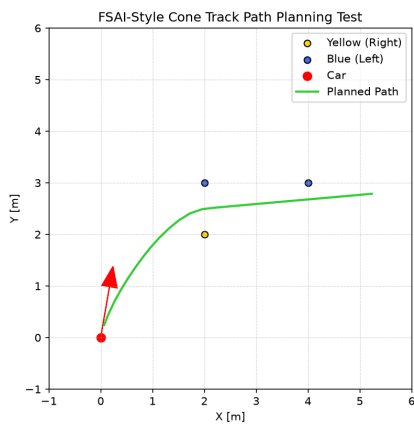


Figure 1

8

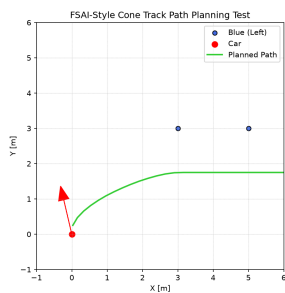
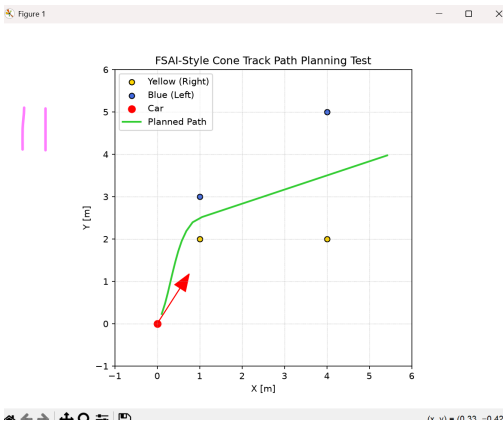
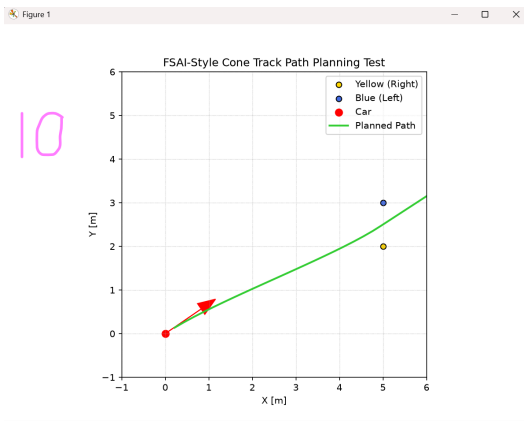
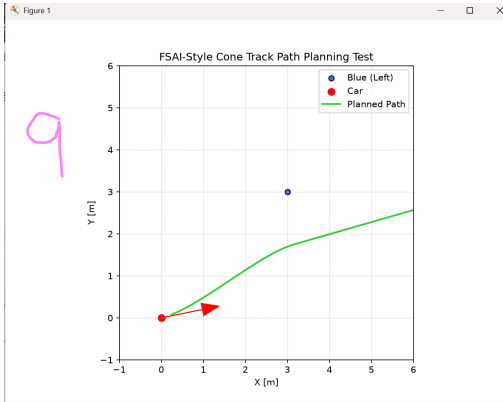
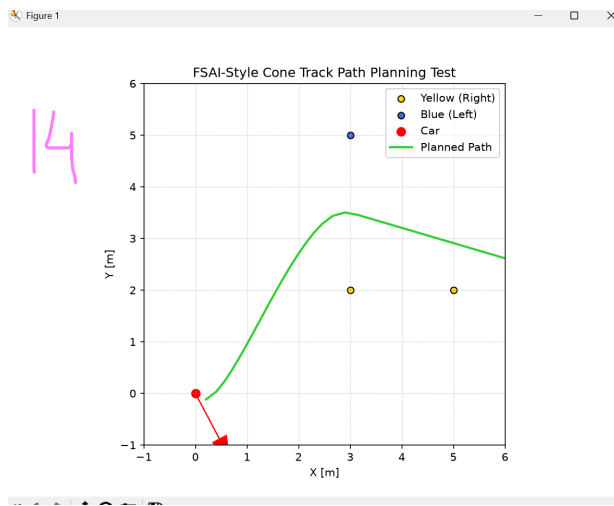
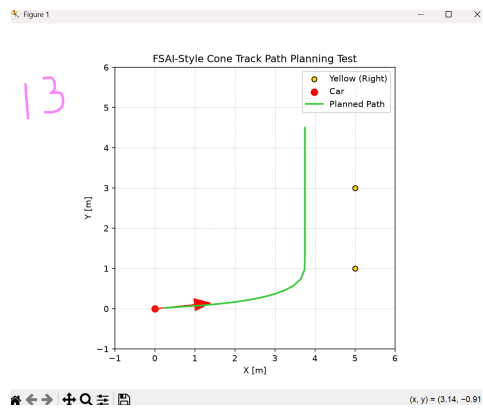
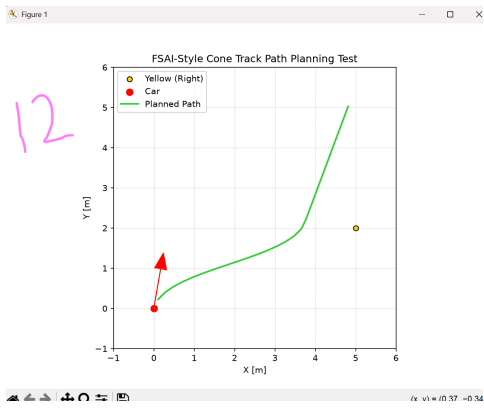
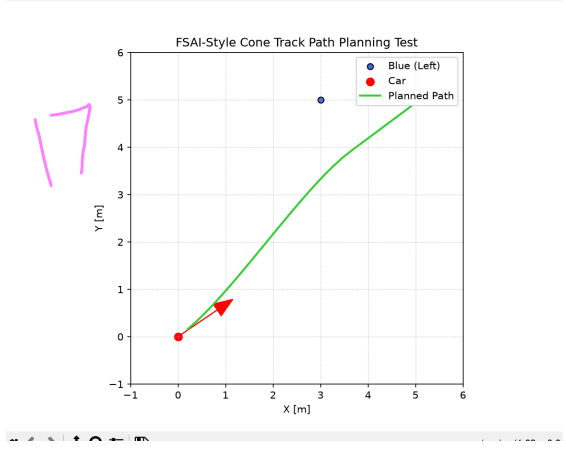
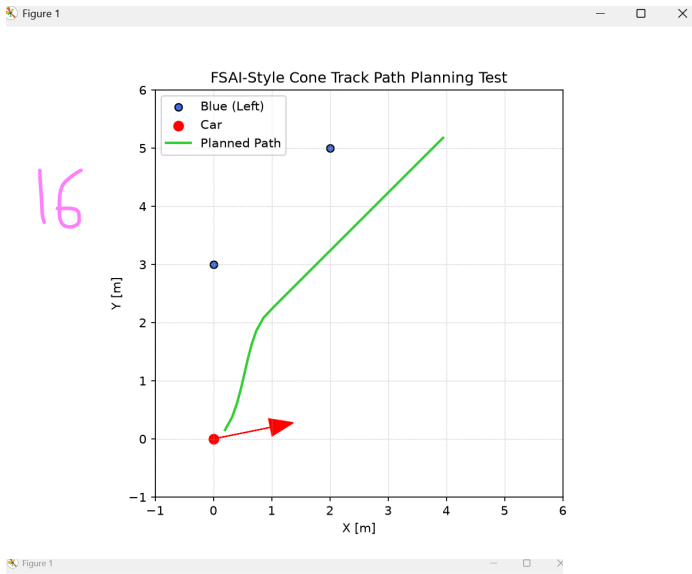
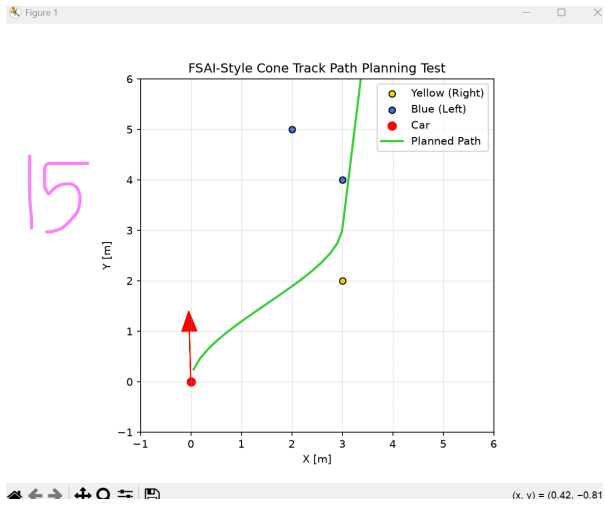
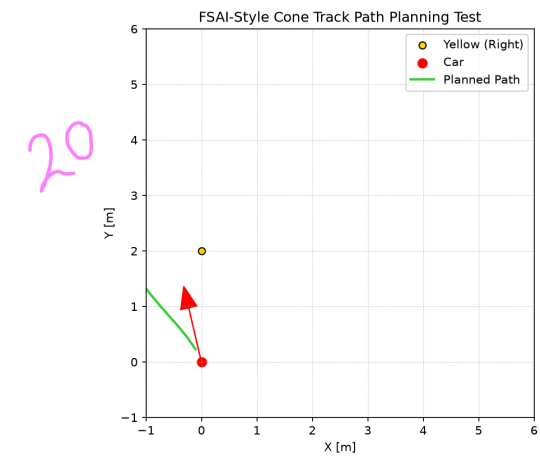
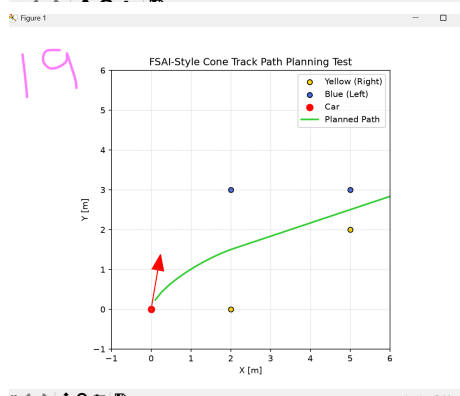
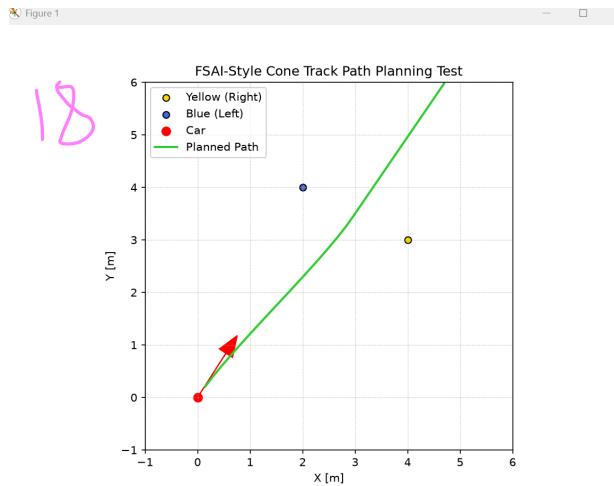


Figure 1









21

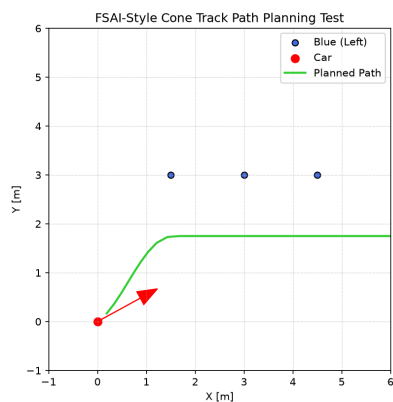


Figure 1

22

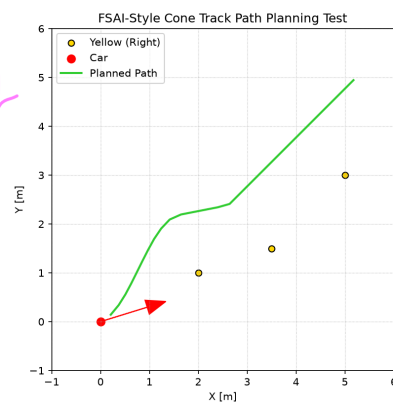
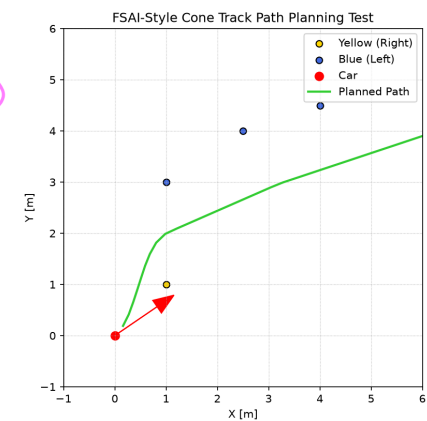
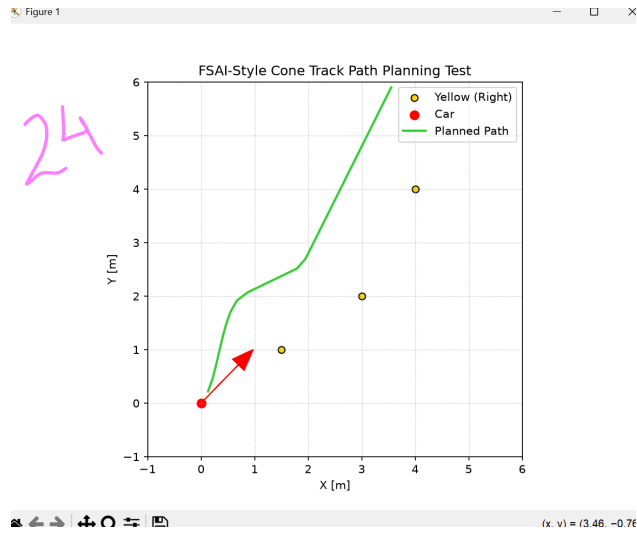


Figure 1

23





Hand Written Notes

path planning task 2

→ pycarum, git, matplotlib ✓

Clarification
also

part (1):

inputs → at most 2 cones with (x, y) coordinates on each

color == 0 yellow right
color == 1 blue left

[FSAI] stuff

→ car pose (x, y, yaw)
car heading

output → sequence of path points (x, y) in world coordinates
drive between blue and yellow cones
and visualize it.

* distance between points $\leq 0.5m$

* path should be 5-10 m long

part (2)

PART ①

SCENARIOS: Dict [str, Tuple [List [core], CarPose]]

→ scenarios are "1" ~ "20"

→ car always at origin (0,0), diff headings

→ value

	cones	car yaw (rad)	test case
"1" ✓	1 •	0	no cones, just st path
"2" ✓	1 •	0.4	get midpoint between two cones
"3" ✓	2 •	0.8	pair → same midpoint
"4" ✓	1 •	1.2	get cone on the right
"5" ✓	2 •	1.6	get 2 cones on the right
"6" ✓	1 •	2	unbalanced sides
"7" ✓	2 •	1.4	unbalanced sides
"8" ✓	2 •	1.8	get cones on the left
"9" ✓	1 •	0.2	get cone on left
"10" ✓	1 •	0.6	cone pair, yaw above
"11" ✓	2 •	1	balanced, diff midpoint
"12" ✓	1 •	1.4	get cone on right
"13" ✓	2 •	0.1	get cones on right (vertical)
"14" ✓	1 •	5.2	unbalanced sides
"15" ✓	2 •	1.6	unbalanced, diff midpoint
"16" ✓	2 •	0.2	get cones on left (diagonal)
"17" ✓	1 •	0.6	get cone on left
"18" ✓	1 •	1	fix yaw
"19" ✓	2 •	1.1	balanced, diff midpoint
"20" ✓	1 •	1.8	get cone on right

Case

No ones

no calculating midpoint
no solve / default path

* one one

keep one on the correct side

one pair

$$\frac{L + R}{2} = P$$

two or more

$$\frac{L_1 + R_1}{2} = P_1$$

$$\frac{L_2 + R_2}{2} = P_2$$

$$P_1 \rightarrow P_2$$

→ confusion
spring

diff. paths



no diff just in middle

balanced pairs

perfect

unbalanced

create virtual ones?

handle pairs and
unwatched pairs
separately

* you don't change

→ not in middle

→ in looking up the path was virtual

→ in range

Improvements ①

Improvements ②

① Calmull-Pore Splice construct

Improvement ③

1	✓	✓	✓
2	✓	no res	✓
3	✓	✓	✓
4	✓	✓	✓
5	??		✓
6	X		X unknown
7			✓
8			✓
9			✓
10			✓
11			✓
12			✓
13			✓
14			X
15	→ (15) ←		✓
16			✓
17			✓
18			✓
19			✓
20			

under construction